순차논리회로 5종 구현 및 검증

RS Latch / D Latch / D Flip-Flop / JK Flip-Flop / Asynchronous Set-Clear D Flip-Flop

1. 과제 목적

게이트 수준 Logisim 회로와 VHDL 구현이 같은 논리 기능을 수행하는지 검증한다.

2. 구현 및 검증 방법

Logisim: 개별 NOR/NAND/NOT 게이트와 Tunnel 배선만 사용. 기본 Latch/Flip-Flop 부품 없음.

VHDL: RTL 소스 5개를 작성하고 GHDL에서 컴파일, 시뮬레이션 및 범용 논리 합성을 수행.

합성된 VHDL 넷리스트도 다시 컴파일하고 동일한 테스트벤치로 재검증했다.

게이트 모델: Logisim과 같은 게이트 연결에 1 ns 지연을 주고 RTL과 출력을 비교했다.

파형은 실제 GHDL VCD에서 생성했고, 회로도는 제출 회로/합성 넷리스트에서 생성했다.

3. 결과 요약

RS Latch	**Logisim 7/7 PASS | RTL / Gates / Synth PASS**
D Latch	**Logisim 8/8 PASS | RTL / Gates / Synth PASS**
Positive-edge D Flip-Flop	**Logisim 10/10 PASS | RTL / Gates / Synth PASS**
Positive-edge JK Flip-Flop	**Logisim 15/15 PASS | RTL / Gates / Synth PASS**
D Flip-Flop with Asynchronous Set/Clear	**Logisim 13/13 PASS | RTL / Gates / Synth PASS**

초기 상태는 보장하지 않는다. RS는 Reset, D Latch는 en=1에서 D=0,
D/JK 플립플롭은 유효한 첫 상승 에지, 비동기 D 플립플롭은 Clear로 초기화한다.

1. RS Latch - 게이트 회로

Logisim gate-level implementation | All matching tunnel labels are electrically connected

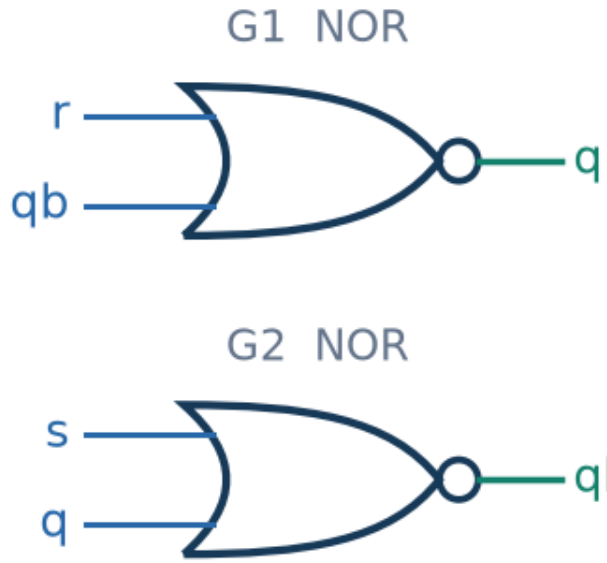
NOR 게이트 2개의 출력을 서로 반대쪽 입력으로 연결하여 상태를 저장한다.

S=1,R=0: Set / S=0,R=1: Reset / S=0,R=0: Hold.

S=R=1에서는 Q=Qb=0이 되어 상보 관계가 깨진다. 금지 상태이다.

금지 상태에서 두 입력을 동시에 해제한 결과는 보장하지 않는다. Reset을 거쳐 복구한다.

IN: s, r OUT: q, qb



Matching net labels are connected (Logisim tunnels), including all feedback paths.

1. RS Latch - VHDL 및 합성

Synthesized schematic generated from GHDL raw netlist | Generic synthesis, not FPGA place-and-route

VHDL RTL

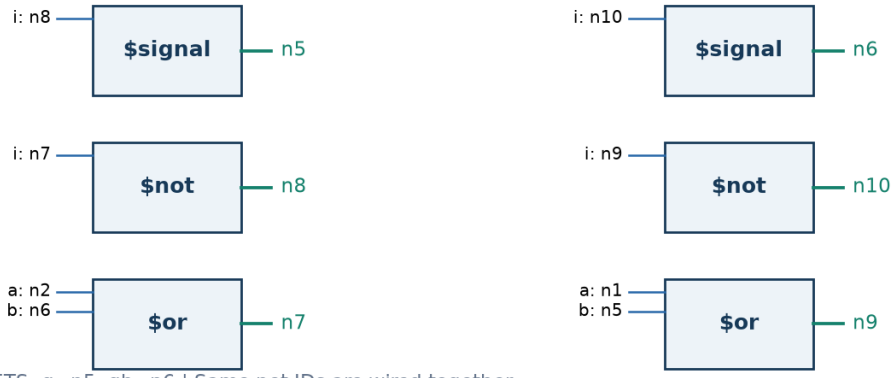
```
library ieee;
use ieee.std_logic_1164.all;

entity rs_latch is
  port (
    s : in std_logic;
    r : in std_logic;
    q, qb : out std_logic
  );
end entity;

architecture rtl of rs_latch is
  signal q_i, qb_i : std_logic;
begin
  -- Active-high NOR latch. s=r=1 forces q=qb=0 (forbidden).
  -- Releasing both controls simultaneously is not a valid stat
  q_i <= not (r or qb_i);
  qb_i <= not (s or q_i);
  q <= q_i; qb <= qb_i;
end architecture;
```

생성된 합성 회로도

INPUT NETS: s=n1, r=n2

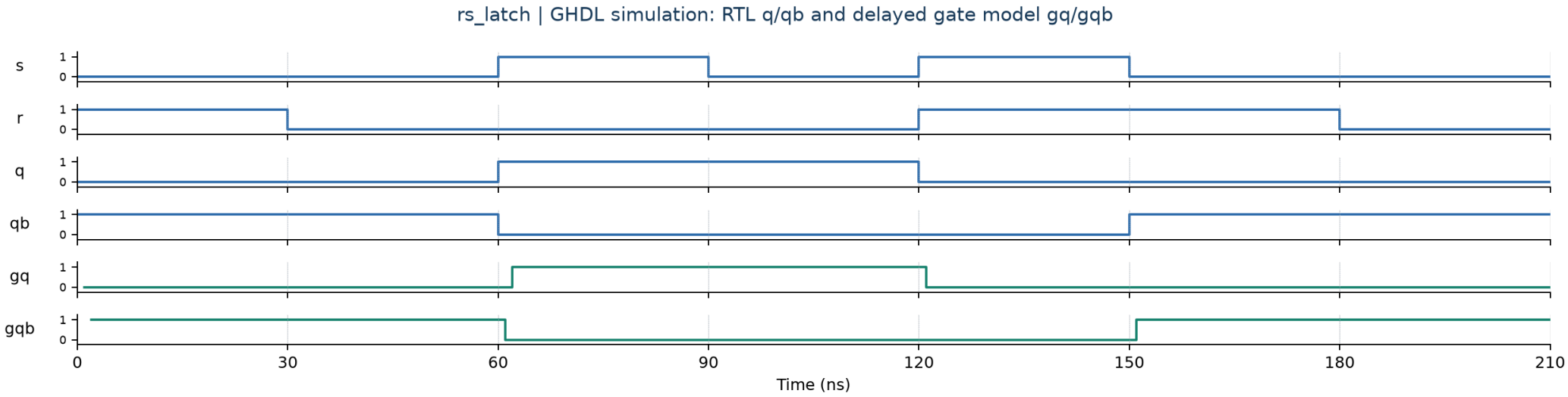


OUTPUT NETS: q=n5, qb=n6 | Same net IDs are wired together.

합성 셀: $dlatch / $dff / $adff 및 조합논리. 출력 Qb는 Q의 반전이며 RS 금지 상태는 예외이다.
넷 ID가 같은 단자는 연결되어 있다. 전체 연결 정보는 synthesis/*_netlist.txt 및 .vhd에 포함했다.

1. RS Latch - 시뮬레이션 결과

Actual VCD waveform | q/qb: RTL outputs, gq/gqb: delayed gate-model outputs



Step	입력 (s, r)	Q / Qb	확인 동작	결과	
1	0, 1		0 / 1	Reset	**PASS**
2	0, 0		0 / 1	Hold 0	**PASS**
3	1, 0		1 / 0	Set	**PASS**
4	0, 0		1 / 0	Hold 1	**PASS**
5	1, 1		0 / 0	Forbidden	**PASS**
6	0, 1		0 / 1	Recover via Reset	**PASS**
7	0, 0		0 / 1	Hold after recovery	**PASS**

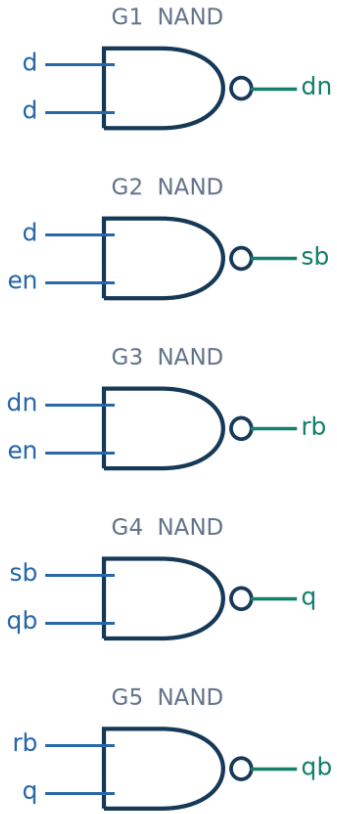
각 단계는 30 ns 유지 후 검사했다. 최초 미초기화 단계는 출력 비교를 제외했다.
1 ns 게이트 지연은 동작 검증용이며 실제 소자의 타이밍/메타안정성을 예측하는 값이 아니다.

2. D Latch - 게이트 회로

Logisim gate-level implementation | All matching tunnel labels are electrically connected

D의 반전과 입력 게이팅, 교차 결합 NAND 쌍으로 총 5개의 NAND 게이트를 사용한다.
en=1에서는 Q가 D를 따라가며, en=0에서는 이전 값을 유지한다.
반전은 NAND의 두 입력을 묶어 구현했다. 기본 Latch 부품은 사용하지 않았다.

IN: d, en OUT: q, qb



Matching net labels are connected (Logisim tunnels), including all feedback paths.

2. D Latch - VHDL 및 합성

Synthesized schematic generated from GHDL raw netlist | Generic synthesis, not FPGA place-and-route

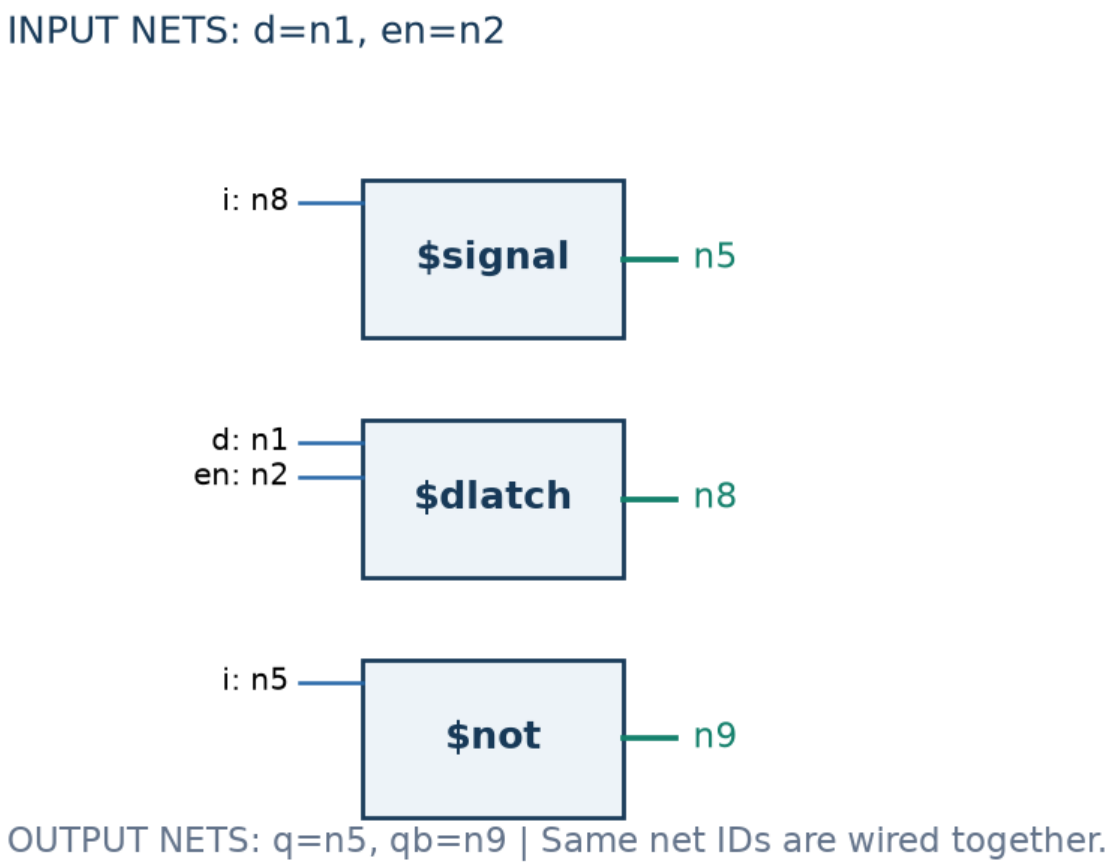
VHDL RTL

```
library ieee;
use ieee.std_logic_1164.all;

entity d_latch is
  port (
    d : in std_logic;
    en : in std_logic;
    q, qb : out std_logic
  );
end entity;

architecture rtl of d_latch is
  signal q_i : std_logic;
begin
  process(d, en)
  begin
    if en = '1' then q_i <= d; end if;
  end process;
  q <= q_i; qb <= not q_i;
end architecture;
```

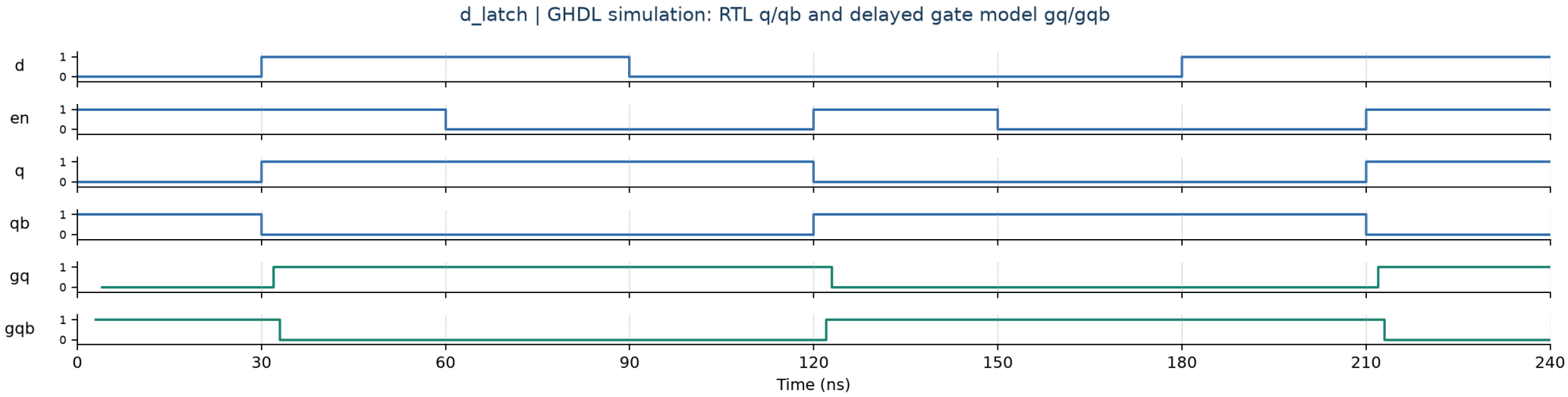
생성된 합성 회로도



합성 셀: $dlatch / $dff / $adff 및 조합논리. 출력 Qb는 Q의 반전이며 RS 금지 상태는 예외이다.
넷 ID가 같은 단자는 연결되어 있다. 전체 연결 정보는 synthesis/*_netlist.txt 및 .vhd에 포함했다.

2. D Latch - 시뮬레이션 결과

Actual VCD waveform | q/qb: RTL outputs, gq/gqb: delayed gate-model outputs



Step	입력 (d, en)	Q / Qb	확인 동작	결과	
1	0, 1		0 / 1	Initialize transparent	PASS
2	1, 1		1 / 0	Follow 1	PASS
3	1, 0		1 / 0	Close latch	PASS
4	0, 0		1 / 0	Hold despite D change	PASS
5	0, 1		0 / 1	Follow 0	PASS
6	0, 0		0 / 1	Close at 0	PASS
7	1, 0		0 / 1	Hold 0	PASS
8	1, 1		1 / 0	Follow again	PASS

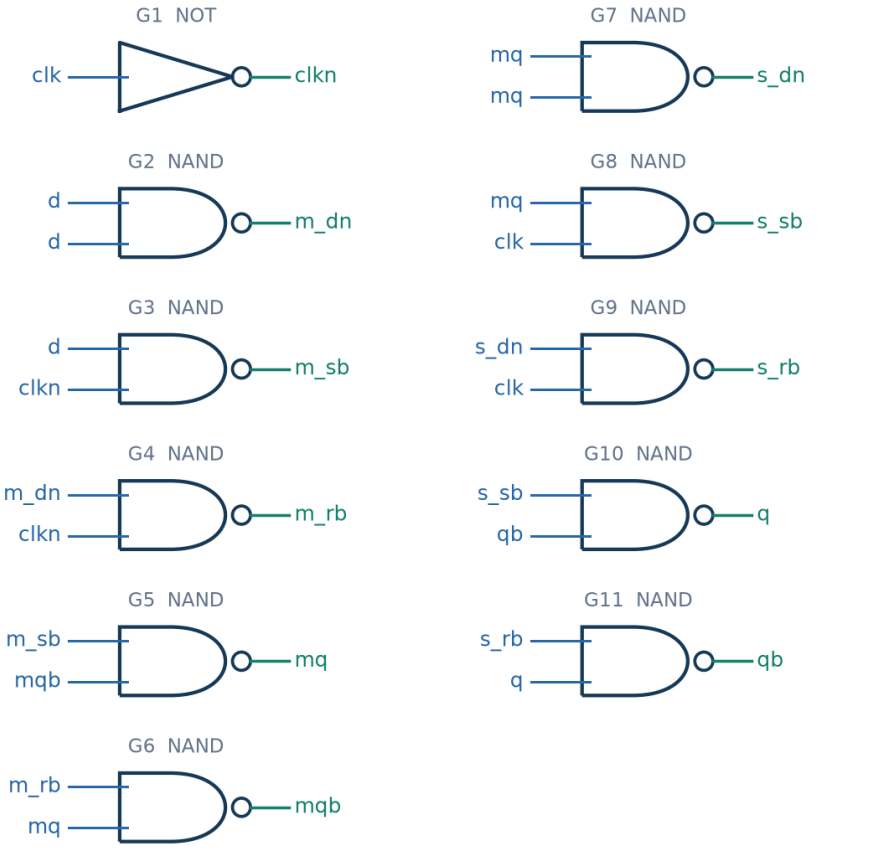
각 단계는 30 ns 유지 후 검사했다. 최초 미초기화 단계는 출력 비교를 제외했다.
1 ns 게이트 지연은 동작 검증용이며 실제 소자의 타이밍/메타안정성을 예측하는 값이 아니다.

3. Positive-edge D Flip-Flop - 게이트 회로

Logisim gate-level implementation | All matching tunnel labels are electrically connected

Master는 clk=0에서 열리고, Slave는 clk=1에서 열린다. 두 래치와 인버터로 구성했다.
clk 상승 에지에서 Master에 저장된 D가 Slave로 전달된다.
D가 클록의 높은 구간 또는 낮은 구간에서 바뀌어도 최종 Q는 다음 상승 에지까지 유지된다.

IN: d, clk OUT: q, qb



Matching net labels are connected (Logisim tunnels), including all feedback paths.

3. Positive-edge D Flip-Flop - VHDL 및 합성

Synthesized schematic generated from GHDL raw netlist | Generic synthesis, not FPGA place-and-route

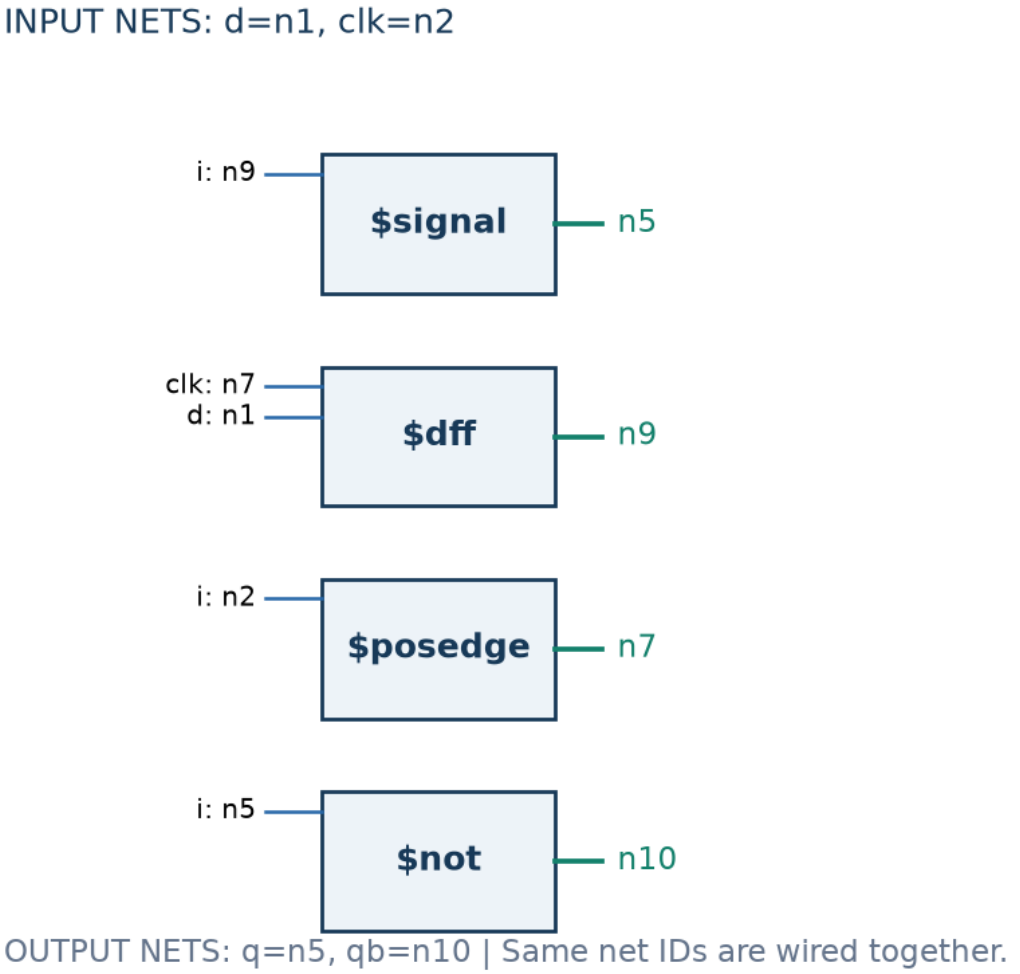
VHDL RTL

```
library ieee;
use ieee.std_logic_1164.all;

entity d_ff is
  port (
    d : in std_logic;
    clk : in std_logic;
    q, qb : out std_logic
  );
end entity;

architecture rtl of d_ff is
  signal q_i : std_logic;
begin
  process(clk)
  begin
    if rising_edge(clk) then q_i <= d; end if;
  end process;
  q <= q_i; qb <= not q_i;
end architecture;
```

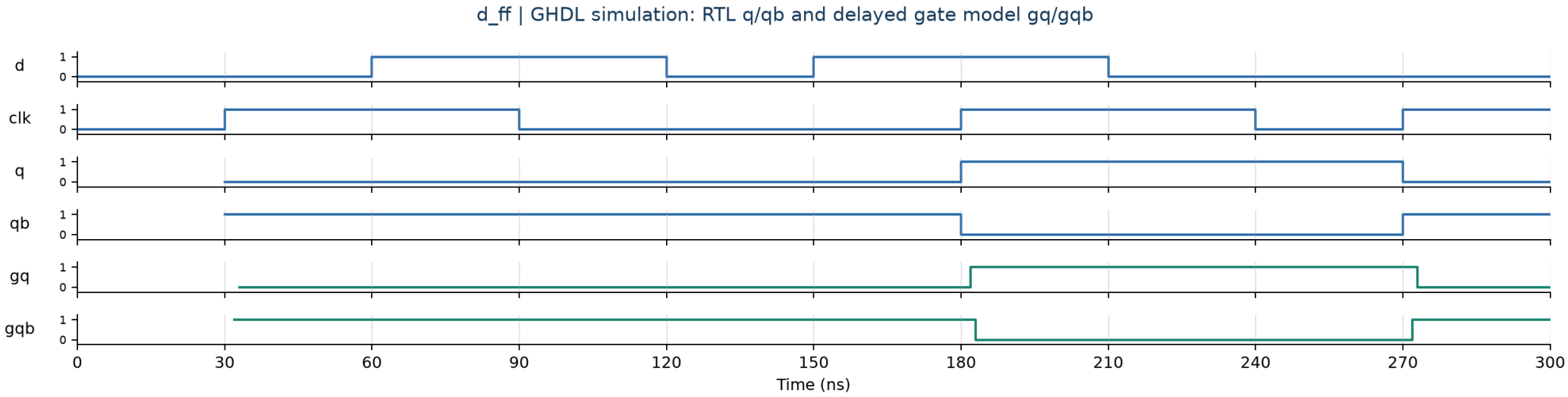
생성된 합성 회로도



합성 셀: $dlatch / $dff / $adff 및 조합논리. 출력 Qb는 Q의 반전이며 RS 금지 상태는 예외이다.
넷 ID가 같은 단자는 연결되어 있다. 전체 연결 정보는 synthesis/*_netlist.txt 및 .vhd에 포함했다.

3. Positive-edge D Flip-Flop - 시뮬레이션 결과

Actual VCD waveform | q/qb: RTL outputs, gq/gqb: delayed gate-model outputs



Step	입력 (d, clk)	Q / Qb	확인 동작	결과	
1	0, 0		unchecked	Prepare D=0	INIT
2	0, 1		0 / 1	Rising captures 0	PASS
3	1, 1		0 / 1	High phase holds	PASS
4	1, 0		0 / 1	Falling holds	PASS
5	0, 0		0 / 1	Low phase change	PASS
6	1, 0		0 / 1	Prepare D=1	PASS
7	1, 1		1 / 0	Rising captures 1	PASS
8	0, 1		1 / 0	High phase holds 1	PASS
9	0, 0		1 / 0	Falling holds 1	PASS
10	0, 1		0 / 1	Rising captures 0	PASS

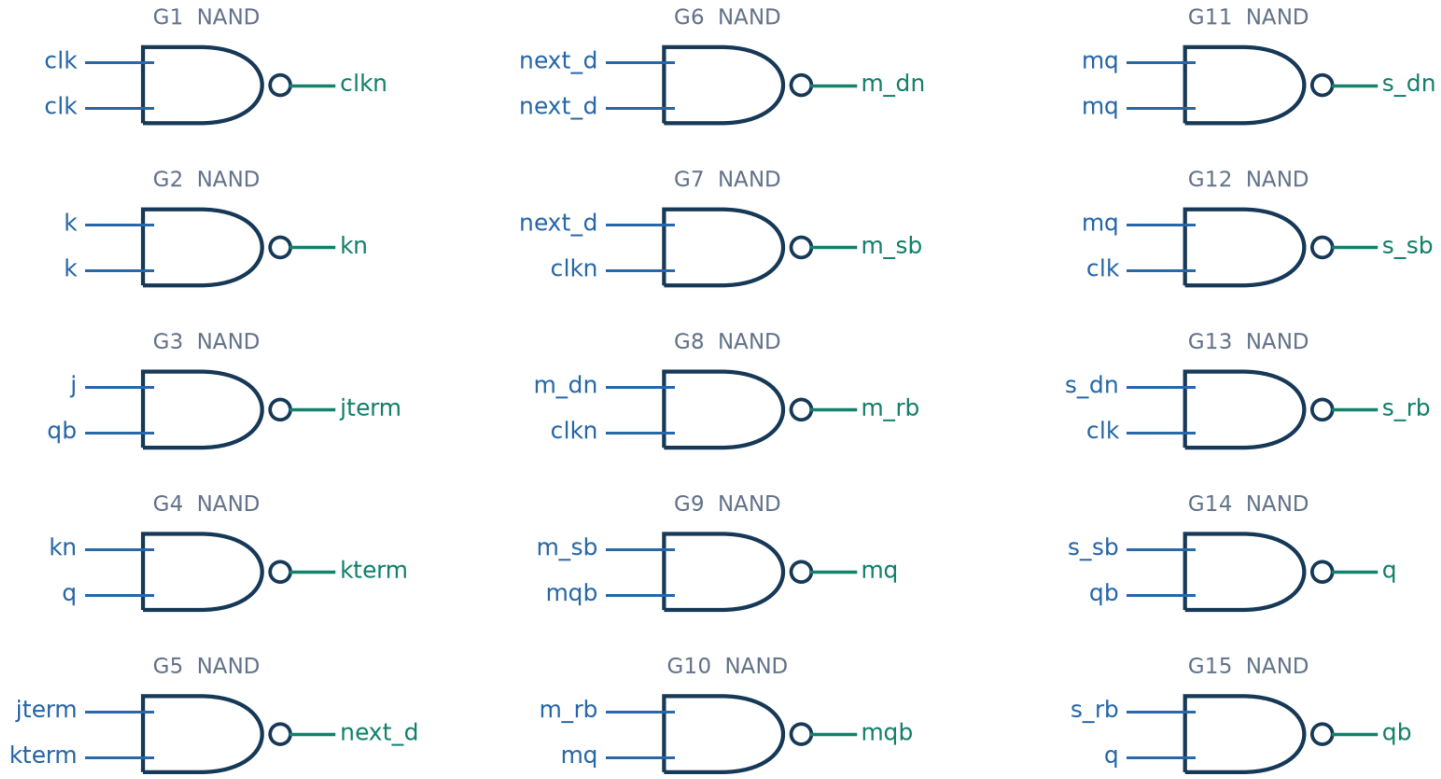
각 단계는 30 ns 유지 후 검사했다. 최초 미초기화 단계는 출력 비교를 제외했다.
1 ns 게이트 지연은 동작 검증용이며 실제 소자의 타이밍/메타안정성을 예측하는 값이 아니다.

4. Positive-edge JK Flip-Flop - 게이트 회로

Logisim gate-level implementation | All matching tunnel labels are electrically connected

다음 상태식 Dnext = J·Qb + (NOT K)·Q를 NAND 게이트로 구현했다.
Q와 Qb를 입력 논리에 피드백하고, NAND Master-Slave 구조로 상승 에지에서만 저장한다.
J,K=00: Hold / 10: Set / 01: Reset / 11: Toggle. clk 반전도 NAND로 구성했다.
Toggle은 에지당 한 번만 발생한다. 레벨 민감 JK 래치의 반복 토글을 피한다.

IN: j, k, clk OUT: q, qb



Matching net labels are connected (Logisim tunnels), including all feedback paths.

4. Positive-edge JK Flip-Flop - VHDL 및 합성

Synthesized schematic generated from GHDL raw netlist | Generic synthesis, not FPGA place-and-route

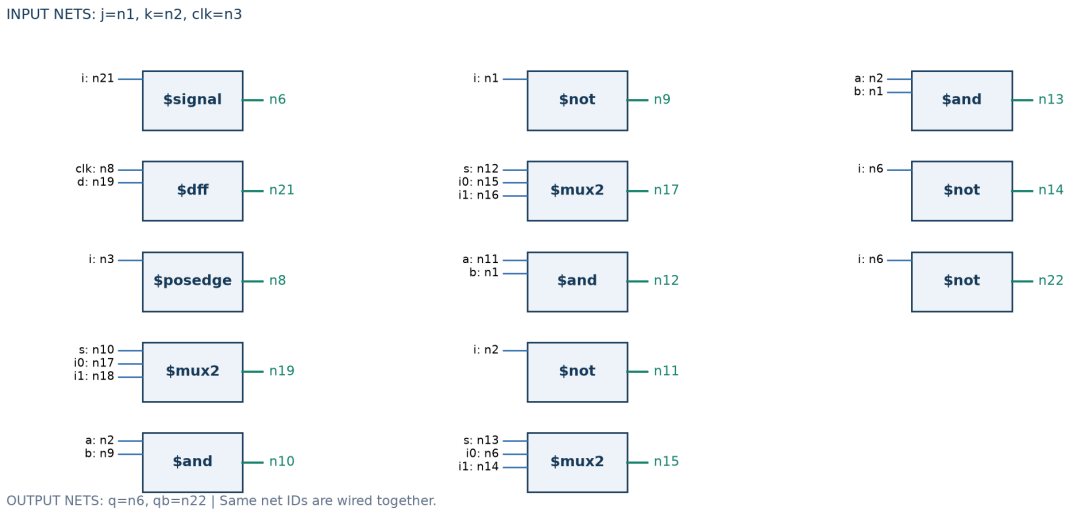
VHDL RTL

```
library ieee;
use ieee.std_logic_1164.all;

entity jk_ff is
  port (
    j : in std_logic;
    k : in std_logic;
    clk : in std_logic;
    q, qb : out std_logic
  );
end entity;

architecture rtl of jk_ff is
  signal q_i : std_logic;
begin
  process(clk)
  begin
    if rising_edge(clk) then
      if j = '0' and k = '1' then q_i <= '0';
      elsif j = '1' and k = '0' then q_i <= '1';
      elsif j = '1' and k = '1' then q_i <= not q_i;
      end if;
    end if;
  end process;
  q <= q_i; qb <= not q_i;
end architecture;
```

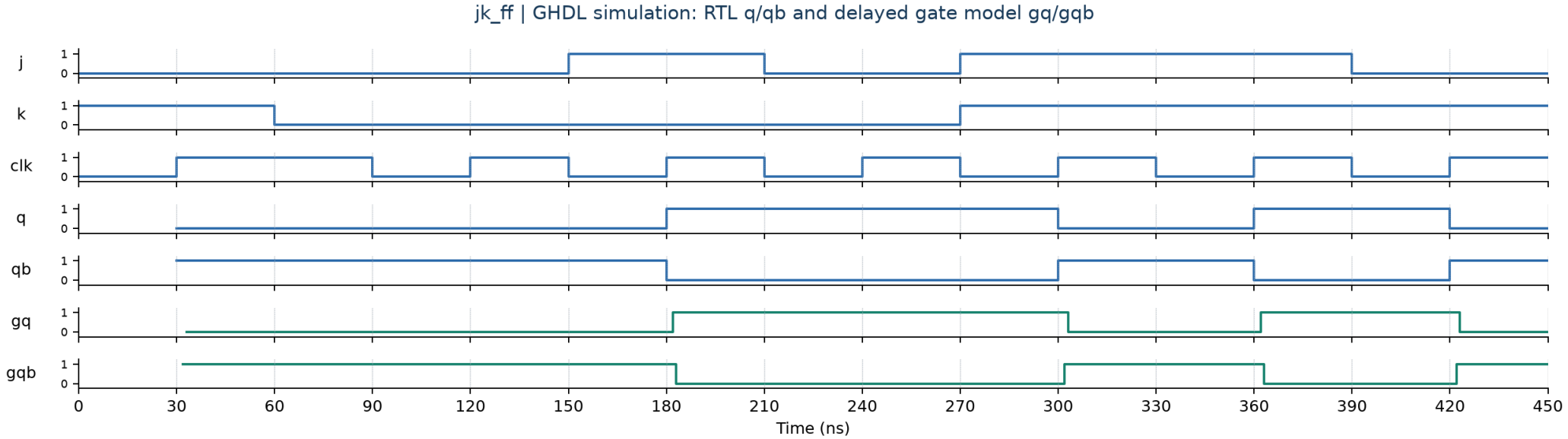
생성된 합성 회로도



합성 셀: $dlatch / $dff / $adff 및 조합논리. 출력 Qb는 Q의 반전이며 RS 금지 상태는 예외이다.
넷 ID가 같은 단자는 연결되어 있다. 전체 연결 정보는 synthesis/*_netlist.txt 및 .vhd에 포함했다.

4. Positive-edge JK Flip-Flop - 시뮬레이션 결과

Actual VCD waveform | q/qb: RTL outputs, gq/gqb: delayed gate-model outputs



Step	입력 (j, k, clk)	Q / Qb	확인 동작	결과	
1	0, 1, 0		unchecked	Prepare Reset	INIT
2	0, 1, 1		0 / 1	Reset	PASS
3	0, 0, 1		0 / 1	High holds	PASS
4	0, 0, 0		0 / 1	Prepare Hold	PASS
5	0, 0, 1		0 / 1	Hold 0	PASS
6	1, 0, 0		0 / 1	Prepare Set	PASS
7	1, 0, 1		1 / 0	Set	PASS
8	0, 0, 0		1 / 0	Prepare Hold 1	PASS
9	0, 0, 1		1 / 0	Hold 1	PASS
10	1, 1, 0		1 / 0	Prepare Toggle	PASS
11	1, 1, 1		0 / 1	Toggle to 0	PASS
12	1, 1, 0		0 / 1	Falling holds	PASS
13	1, 1, 1		1 / 0	Toggle to 1	PASS
14	0, 1, 0		1 / 0	Prepare Reset again	PASS
15	0, 1, 1		0 / 1	Reset again	PASS

각 단계는 30 ns 유지 후 검사했다. 최초 미초기화 단계는 출력 비교를 제외했다.

1 ns 게이트 지연은 동작 검증용이며 실제 소자의 타이밍/메타안정성을 예측하는 값이 아니다.

5. D Flip-Flop with Asynchronous Set/Clear - 게이트 회로

Logisim gate-level implementation | All matching tunnel labels are electrically connected

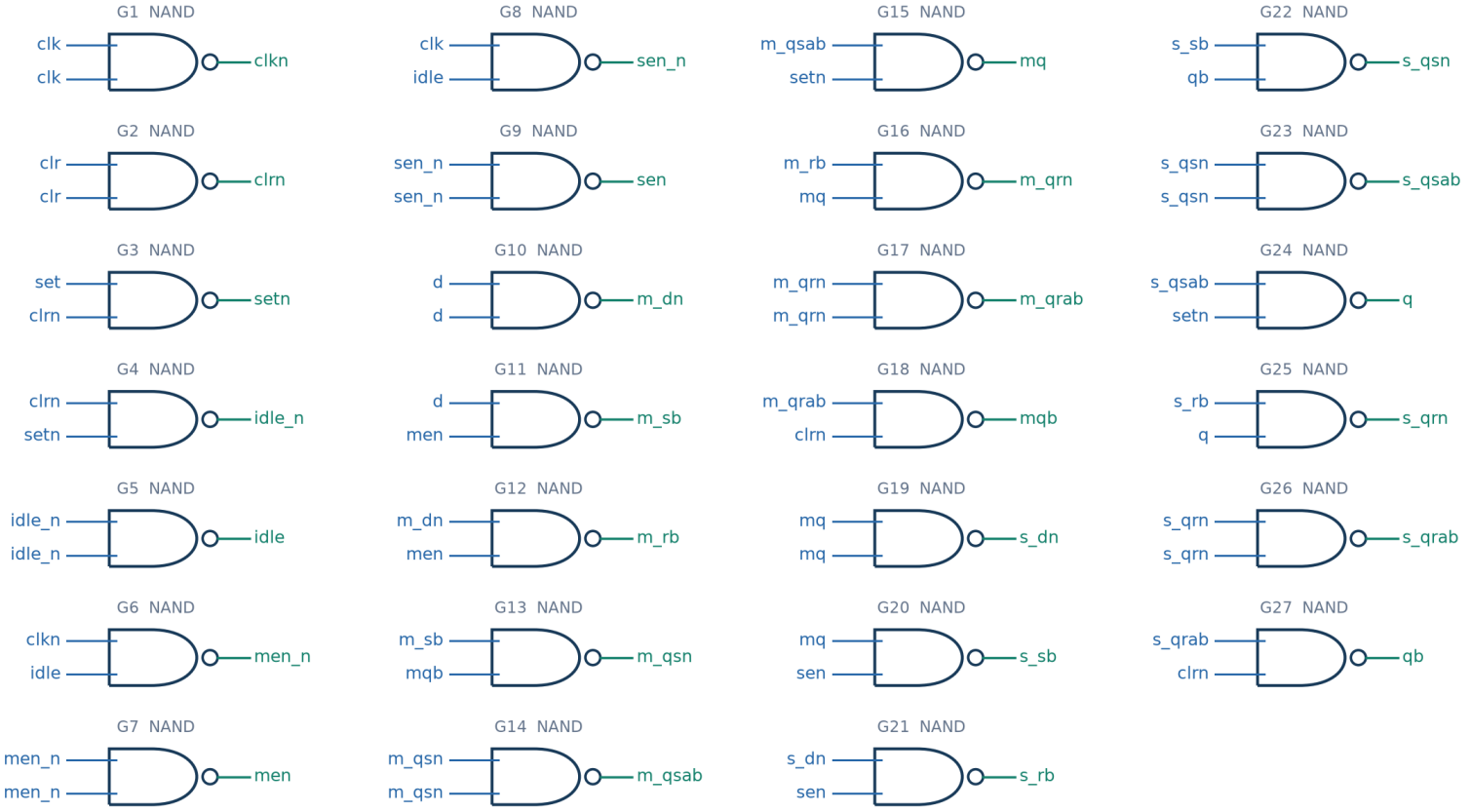
Clear를 먼저 적용하고, Set은 Clear가 0일 때만 유효하도록 구성했다.

clrn=NOT clr, setn=NOT(set AND clrn), idle=clrn AND setn.

비동기 제어 중에는 데이터 경로를 차단하고 Master와 Slave 모두에 동일한 제어를 적용한다.

Clear와 Set은 클록 에지를 기다리지 않는다. 두 신호가 0이면 정상 D 플립플롭으로 동작한다.

IN: d, clk, set, clr OUT: q, qb



Matching net labels are connected (Logisim tunnels), including all feedback paths.

5. D Flip-Flop with Asynchronous Set/Clear - VHDL 및 합성

Synthesized schematic generated from GHDL raw netlist | Generic synthesis, not FPGA place-and-route

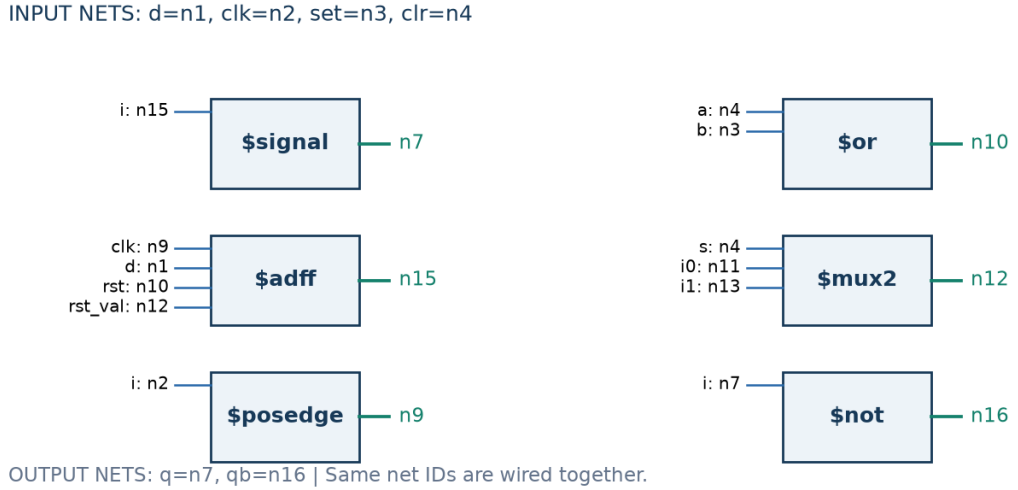
VHDL RTL

```
library ieee;
use ieee.std_logic_1164.all;

entity d_ff_async is
  port (
    d : in std_logic;
    clk : in std_logic;
    set : in std_logic;
    clr : in std_logic;
    q, qb : out std_logic
  );
end entity;

architecture rtl of d_ff_async is
  signal q_i : std_logic;
begin
  process(clk, set, clr)
  begin
    if clr = '1' then q_i <= '0';
    elsif set = '1' then q_i <= '1';
    elsif rising_edge(clk) then q_i <= d;
    end if;
  end process;
  q <= q_i; qb <= not q_i;
end architecture;
```

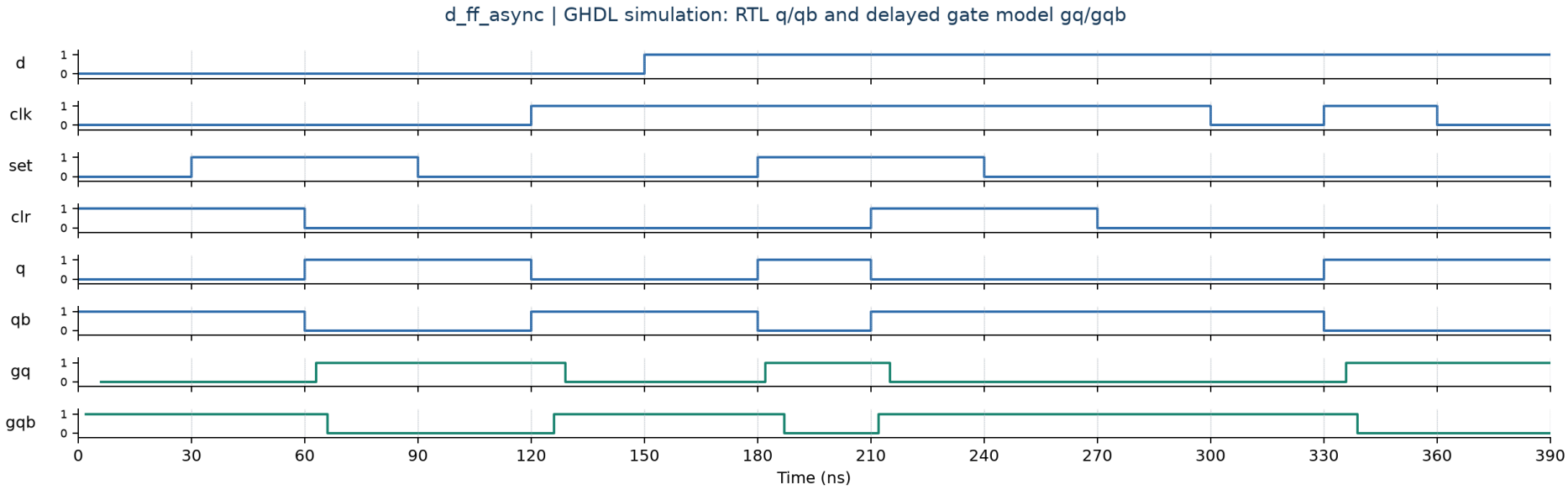
생성된 합성 회로도



합성 셀: $dlatch / $dff / $adff 및 조합논리. 출력 Qb는 Q의 반전이며 RS 금지 상태는 예외이다.
넷 ID가 같은 단자는 연결되어 있다. 전체 연결 정보는 synthesis/*_netlist.txt 및 .vhd에 포함했다.

5. D Flip-Flop with Asynchronous Set/Clear - 시뮬레이션 결과

Actual VCD waveform | q/qb: RTL outputs, gq/gqb: delayed gate-model outputs



Step	입력 (d, clk, set, clr)	Q / Qb	확인 동작	결과
1	0, 0, 0, 1	0 / 1	Async Clear at low	PASS
2	0, 0, 1, 1	0 / 1	Clear wins both high	PASS
3	0, 0, 1, 0	1 / 0	Async Set no edge	PASS
4	0, 0, 0, 0	1 / 0	Release Set holds	PASS
5	0, 1, 0, 0	0 / 1	Normal rising captures 0	PASS
6	1, 1, 0, 0	0 / 1	High D change holds	PASS
7	1, 1, 1, 0	1 / 0	Async Set while high	PASS
8	1, 1, 1, 1	0 / 1	Async Clear overrides Set	PASS
9	1, 1, 0, 1	0 / 1	Clear maintained	PASS
10	1, 1, 0, 0	0 / 1	Release Clear no edge	PASS
11	1, 0, 0, 0	0 / 1	Falling holds	PASS
12	1, 1, 0, 0	1 / 0	Normal rising captures 1	PASS
13	1, 0, 0, 0	1 / 0	Return low	PASS

각 단계는 30 ns 유지 후 검사했다. 최초 미초기화 단계는 출력 비교를 제외했다.
1 ns 게이트 지연은 동작 검증용이며 실제 소자의 타이밍/메타안정성을 예측하는 값이 아니다.

재현 방법 및 제출 파일

Reproducible verification and deliverables

1. Logisim

logisim/sequential_logic.circ를 열고 왼쪽에서 원하는 회로를 선택한다.

각 회로의 입력 Pin을 Poke 도구로 클릭한다. tests/*.txt 순서로 입력을 적용한다.

Simulate > Test Vector에서 해당 txt 파일을 불러오면 순차 테스트를 실행할 수 있다.

이 회로는 게이트만 사용하므로 VHDL Simulation Enabled 또는 Questa 연동이 필요 없다.

2. VHDL 검증 및 합성

GHDL 6.0.0과 Java 21 이상을 준비한 뒤 README의 실행 명령을 사용한다.

verify.py는 RTL, 1 ns 게이트 모델, 합성 넷리스트, Logisim 순차 벡터를 검증한다.

results/에는 로그와 VCD 파형, synthesis/에는 생성된 범용 합성 넷리스트가 있다.

3. 제출 구성

vhdl/*.vhd / testbench/*.vhd / logisim/*.circ / output/pdf/*.pdf

추가 검증 자료: tests/, results/, synthesis/, figures/, README.md.

참고 도구 문서

GHDL: https://ghdl.github.io/ghdl/using/Synthesis.html

Logisim Test Vector: https://github.com/logisim-evolution/logisim-evolution/blob/main/docs/test_vector.md